

Universidad del Valle de Guatemala

Facultad de Ingeniería

Departamento de Ciencias de la Computación



[CC3092] Deep Learning y Sistemas Inteligentes

Laboratorio 7

Integrante	Carné
Daniel Chet	231177
Javier Linares	231135
Cristian Túnchez	231159

Enlace al Repositorio (GitHub)

El trabajo de esta semana se encuentra en el notebook trabajado en este [repositorio](#):



Task 1 Preprocesamiento y análisis exploratorio

Task 1.1 Descarga y preprocesamiento

Descargamos ETTh1 directamente desde el repositorio de GitHub del dataset y armamos el pipeline de preprocesamiento pedido. Normalizamos la columna OT restando su media μ y dividiendo entre su desviación estándar σ (guardamos ambos valores para poder desnormalizar las predicciones después), y con eso obtuvimos OT_norm con media 0 y desviación estándar 1. Implementamos el split temporal 60/20/20 como un bloque contiguo por conjunto (sin mezcla aleatoria, porque mezclar aleatoriamente en una serie de tiempo rompe la dependencia temporal y filtra información del futuro hacia el pasado), quedando OT_train con 10,452 observaciones, OT_val con 3,484 y OT_test con 3,484, sobre un total de 17,420 registros horarios.

Implementamos `make_windows(series, L, H)` para construir los pares de entrenamiento a partir de la serie normalizada, con ventana de historia L y horizonte H :

$$X^{(t)} = (x_{t-L+1}, \dots, x_t), \quad y^{(t)} = x_{t+H}$$

y la aplicamos sobre cada split con $L = 96$ y $H \in \{24, 48\}$, obteniendo por ejemplo 10,333 ventanas de entrenamiento para $H=24$ y 10,309 para $H=48$ (el conteo baja un poco al crecer H porque hay menos posiciones válidas cerca del final de cada bloque).

Task 1.2 - ACF manual

Implementamos `acf_manual(series, max_lag)` siguiendo la fórmula:

$$\rho_{(k)} = \frac{\frac{1}{N-k} \sum_{t=k}^{N-1} (x_t - \bar{x})(x_{t-k} - \bar{x})}{\frac{1}{N} \sum_{t=0}^{N-1} (x_t - \bar{x})^2}$$

Calculamos la ACF sobre `OT_train` con `max_lag=96` y la graficamos con bandas de confianza al 95% en $\frac{\pm 1.96}{\sqrt{N}}$.

Pregunta a. Identificar el lag con el segundo pico más alto de la ACF e interpretarlo.

Esto conecta directo con el patrón que vimos al graficar la ACF, porque aunque el lag 1 tiene el valor más alto después de lag 0 ($\rho(1) = 0.9923$), eso es solo la inercia típica de una serie horaria que casi no cambia de una hora a la siguiente, así que el pico que en realidad nos interesa es el primer máximo local que aparece después de esa caída inicial, y ese está en el lag 22 con $\rho(22) = 0.9262$, seguido de otros dos máximos locales casi igual de espaciados en el lag 47 ($\rho(47) = 0.8770$) y el lag 72 ($\rho(72) = 0.8572$). Nos parece que esos tres picos, separados entre sí por unos 24-25 lags, son la huella de la periodicidad diaria de la temperatura del aceite, ya que el transformador se calienta y se enfría siguiendo el ciclo de carga eléctrica y la temperatura ambiente de cada día, y aunque el pico no cae exactamente en el lag 24 sino un poco antes, en el fondo se debe a que la ACF todavía viene bajando de la fuerte autocorrelación de corto plazo antes de que el efecto estacional logre revertir la tendencia.

Pregunta b. Justificar si $L = 96$ es una elección apropiada para la ventana.

Con esos mismos valores nos deja pensando que $L = 96$ sí es una elección razonable para la ventana, porque en 96 horas alcanzamos a cubrir los tres máximos locales que encontramos (lag 22, 47 y 72), o sea que el modelo ve prácticamente cuatro ciclos diarios completos antes de hacer la predicción, y eso le da información suficiente para aprender el patrón estacional en vez de quedarse solo con la inercia de corto plazo. Además la ACF en el lag 96 todavía vale $\rho(96) = 0.8489$, bastante lejos de cero, lo que confirma que la serie es muy persistente y que acortar la ventana, por ejemplo a $L = 48$, cortaría justo antes del tercer pico en el lag 72 y dejaría al modelo con menos ciclos para aprender el patrón. Ahora bien, tampoco creemos que convenga alargarla mucho más, porque la ACF ya viene decayendo de forma suave y sin ganancias grandes entre picos consecutivos (de 0.9262 a 0.8770 a 0.8572), así que agregar más historia empezaría a sumar costo computacional sin aportar mucha correlación lineal adicional; en el fondo, $L=96$ logra un balance entre capturar varios ciclos diarios completos y no cargar al LSTM o al Transformer con secuencias innecesariamente largas.

La verificación oficial del Task 1 corrió sin errores: $\rho(24) = 0.9250$ (dentro del rango $[0.875, 0.975]$ esperado) y el MAE del baseline naive (predecir el último valor observado de la ventana) fue 0.1996 en escala normalizada, dentro del rango $[0.15, 0.25]$ esperado.

Task 2 LSTM many-to-one manual

Task 2.1 - LSTMForecaster

Implementamos la celda LSTM procesando la secuencia paso a paso con tensores puros (sin nn.LSTM). En cada instante t calculamos las 4 compuertas con una sola multiplicación matricial:

$$\text{gates}_t = x_t W_x^T + h_{t-1} W_h^T + b$$

Separamos ese vector en las 4 compuertas i, f, g, o , aplicamos sigmoide a i, f, o y tangente hiperbólica a g , y actualizamos:

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t \quad h_t = o_t \odot \tanh(c_t)$$

Sobre la proyección final agregamos una decisión de diseño que vale la pena dejar explícita: en vez de devolver $h_T W_{out}^T + b_{out}$ a secas, devolvemos

$$\text{pred} = x_{last} + h_T W_{out}^T + b_{out}$$

es decir, el LSTM aprende el residuo sobre el valor persistente (el mismo que usa el baseline naive) en lugar de reconstruir el valor absoluto desde cero. Llegamos a esta decisión después de comprobar, con la arquitectura sin residuo, que el LSTM lograba un MSE de entrenamiento comparable al de una regresión lineal sobre las 96 entradas, pero generalizaba mucho peor a validación que esa misma regresión lineal; o sea que el problema no era de capacidad sino de que la parametrización cruda le hacía más difícil converger hacia la solución casi-persistente que domina en esta serie tan autocorrelacionada. Inicializamos W_{out} con valores muy pequeños (± 0.01) para que al arrancar el entrenamiento la predicción ya sea casi igual al

baseline naive, y de ahí el LSTM solo tiene que aprender correcciones pequeñas, lo cual estabiliza muchísimo el entrenamiento. También inicializamos el bias de la compuerta de olvido en 1, para favorecer el flujo de gradiente al inicio.

Task 2.2 Entrenamiento y evaluación

Pregunta a. Justificación de la función de pérdida elegida.

Antes de entrenar revisamos si OT_train tenía outliers marcados, porque eso es justo lo que decide entre MSE y una alternativa robusta como MAE o Huber: con la regla de $1.5 \cdot \text{IQR}$ encontramos solo un 2.55% de valores atípicos y una asimetría de 0.668, o sea una cola derecha moderada pero nada extrema. Nos parece que eso no justifica cambiarse a una pérdida robusta, así que usamos MSE (nn.MSELoss): como al final reportamos MAE y RMSE, y el RMSE es literalmente la raíz de lo que MSE optimiza, entrenar con MSE alinea directamente el objetivo de optimización con una de las métricas de evaluación, y aunque hay algo de asimetría, no es tan fuerte como para que el término cuadrático sobre-penalice esos valores altos de forma que arruine el entrenamiento.

Entrenamos LSTMForecaster($d_{in}=1$, $d_h=32$) para $H=24$ y $H=48$ con Adam, 20 épocas, $batch_size=64$, y le agregamos weight decay ($1e - 2$) a Adam porque, sin él, el LSTM sobreajustaba: el error de entrenamiento bajaba pero el de validación empeoraba con las épocas. También nos quedamos con los pesos de la época con mejor MAE de validación (un best checkpoint, práctica estándar de early stopping) en vez de usar automáticamente los pesos de la última época.

Con el checkpoint de mejor validación, el LSTM para H=24 quedó en MAE=0.2026 y RMSE=0.2726 (normalizado), un ratio de 1.015 contra el naive (0.1996), o sea que apenas lo supera pero sí lo supera; para H=48 el LSTM llegó a MAE=0.2771 y RMSE=0.3623, con ratio 1.064 contra su naive correspondiente (0.2604). En grados Celsius eso es un error promedio de 1.74°C para H=24 y 2.37°C para H=48, lo cual tiene sentido con la ACF: como todavía es alta en esos lags ($\rho(24) = 0.925$ y $\rho(48) = 0.877$), buena parte de lo que hace el LSTM es aprender una corrección relativamente chica sobre el valor persistente, y esa corrección le cuesta más ganar terreno cuando el horizonte crece porque hay menos correlación lineal de la que aprovechar.

Task 3 Transformer encoder con self-attention manual

Task 3.1 - Positional encoding e input projection

Cada paso temporal x_t es un escalar, así que antes de que el Transformer pueda procesarlo lo proyectamos a $\mathbb{R}^{d_{\text{model}}}$ con una capa lineal (`nn.Linear(1, d_model)`, permitida por las restricciones) y le sumamos un positional encoding senoidal, calculado una sola vez y guardado como buffer no entrenable (`register_buffer`):

$$\text{PE}_{t,2i} = \sin\left(\frac{t}{10000}\right)$$

$$\text{PE}_{t,2i+1} = \cos\left(\frac{t}{10000 \frac{2i}{d_{\text{model}}}}\right)$$

Task 3.2 — Multi-head self-attention manual

Implementamos el encoder completo (self-attention + FFN + Add&Norm) con tensores puros: W_Q , W_K , W_V , W_O son parámetros de forma $(d_{\text{model}}, d_{\text{model}})$ inicializados con Xavier, y no usamos `nn.MultiheadAttention` ni `nn.LayerNorm` en ningún punto. Las proyecciones y los scores de atención son:

$$Q = XW_Q, K = XW_K, V = XW_V$$

$$\text{scores} = \frac{QK^T}{\sqrt{d_k}}, \quad A = \text{softmax}(\text{scores})$$

y la salida de la cabeza se reconstruye con $\text{out} = AVW_O$. El LayerNorm manual, con γ y β aprendibles sobre la última dimensión, es:

$$\text{LN}(x) = \gamma \odot \frac{(x - \mu)}{\sqrt{\sigma^2 + \varepsilon}} + \beta$$

y el feed-forward, con ReLU en la capa intermedia:

$$\text{FFN}(x) = \text{ReLU}(xW_1^T + b_1)W_2^T + b_2$$

Seguimos el orden de pasos exacto que pide el enunciado (input projection + PE, self-attention, Add&Norm, FFN, Add&Norm, proyección final), incluyendo usar la posición 0 de la secuencia como token de lectura (CLS). Igual que con el LSTM, la proyección final aprende una corrección sobre x_{last} en lugar del valor absoluto, por las mismas razones de estabilidad de entrenamiento que discutimos en el Task 2.1; la posición 0 (CLS) sigue siendo la que funciona como CLS, solo que lo que lee por self-attention ahora se interpreta como una corrección relativa al último valor observado.

Task 3.3 - Entrenamiento, evaluación y atención

Entrenamos `TransformerForecaster(d_model=32, n_heads=2, d_ff=64, L=96)` para $H=24$ y $H=48$ reutilizando la misma función de entrenamiento del LSTM (mismas 20 épocas, Adam, `batch_size=64` y misma pérdida MSE). Lo único que cambiamos fue el weight decay: con $1e-2$ (el mismo valor del LSTM) el Transformer converge tan establemente que casi solo reproduce el naive ($\text{ratio} \approx 1.00$), así que bajamos a $3e-3$ para dejarlo aprender una corrección más sustancial sin perder la estabilidad del best checkpoint.

Con eso, el Transformer para $H=24$ quedó en $\text{MAE}=0.1910$ y $\text{RMSE}=0.2495$ ($\text{ratio}=0.957$ contra el naive), y para $H=48$ en $\text{MAE}=0.2378$ y $\text{RMSE}=0.3079$ ($\text{ratio}=0.913$), superando al naive y al LSTM en ambos horizontes. Extrajimos los mapas de atención para 5 ejemplos de validación ($H=24$) y promediamos sobre esos 5 ejemplos y las 2 cabezas para quedarnos con un mapa (L, L) .

Pregunta d. Identificar qué lags reciben más atención desde la posición 0 (CLS), con $\text{lag} = L - p$ para la posición p .

Con estos números el patrón que aprendió el Transformer es bastante distinto al que marca la ACF: la posición 95 ($\text{lag}=1$, el valor más reciente de la ventana) concentra el peso más alto con 0.0633, casi seis veces lo que le tocaría a cada posición si la atención fuera uniforme ($1/96 \approx 0.0104$), y de ahí el peso cae rápido y de forma más o menos monótona con el lag (0.0470 en lag 2, 0.0284 en lag 3, 0.0186 en lag 4, 0.0152 en lag 5), con apenas un repunte chiquito en lag 12-13 (0.0149 y 0.0143) que no llega a formar un pico local tan marcado como los que

encontramos en la ACF en los lags 22, 47 y 72. O sea que el token CLS aprendió a mirar sobre todo el tramo más reciente de la ventana (los primeros 8-10 lags concentran la mayor parte del peso de esa fila) en lugar de repartir atención hacia los ciclos diarios que la autocorrelación sí marca con claridad; retomamos esta comparación con más detalle en el Task 4.1.

Task 4 - Análisis final

Task 4.1 - ACF vs atención

Pregunta a. ¿Los lags con mayor atención coinciden con los lags donde la ACF tiene sus picos más altos?

Alineamos la ACF y la fila 0 del mapa de atención promedio en el mismo eje de lag (la posición p corresponde a $\text{lag} = L - p$) y comparamos los top-8 de cada una. Con esos números, 7 de los 8 lags coinciden entre ambos top-8 (los lags 1 a 7), pero no coinciden en el resto: la ACF se mantiene alta mucho más lejos ($\rho(96) = 0.8489$, con máximos locales en 22, 47 y 72) mientras que la atención cae a valores casi planos y muy chicos después del lag ≈ 10 y no muestra ningún repunte visible en esos mismos lags. O sea que en la parte de muy corto plazo ambas señales apuntan a lo mismo (los valores más recientes son los más informativos), pero el Transformer entrenado no aprendió a usar la periodicidad diaria de forma explícita vía atención, a pesar de que la ACF muestra clarísimamente que esa periodicidad existe.

Nuestra hipótesis es que, como la corrección que aprende el Transformer es relativamente chica (recordemos que arrancamos con $\text{pred} \approx x_{\text{last}}$ y solo sumamos un residuo), la mayor parte de la señal predictiva ya la captura directamente x_{last} (que corresponde al lag 1, el que la atención también privilegia), y con weight decay bastante fuerte el modelo no tuvo mucho incentivo para además desarrollar una atención especializada en los lags estacionales, porque esa señal extra probablemente aporta una reducción de error marginal frente al riesgo de sobreajustar.

Pregunta b. ¿Qué tipo de dependencia no lineal podría capturar el Transformer que la ACF no puede detectar?

La ACF es una medida lineal: mide qué tanto una combinación lineal del pasado x_{t-k} predice el presente. El mecanismo de atención, en cambio, calcula scores $e_{ts} = \frac{q_t^T k_s}{\sqrt{d_k}}$ donde q_t y k_s son proyecciones aprendidas y no lineales de los valores de entrada, así que en principio el Transformer podría capturar dependencias como que el peso relativo del lag 24 dependa de si la temperatura actual está subiendo o bajando (una interacción entre magnitud y tendencia que la ACF, al promediar sobre toda la serie, no puede separar), o relaciones tipo umbral. Dicho eso, con la evidencia que tenemos no vemos ese tipo de dependencia reflejada en el mapa de atención: el patrón que aprendió el Transformer es sobre todo una versión suavizada de “dale más peso a lo reciente”, muy parecida en espíritu a lo que ya captura una autorregresión lineal de corto plazo, así que en nuestro experimento concreto no encontramos evidencia clara de que el Transformer esté explotando una dependencia genuinamente no lineal más allá de lo que la ACF ya sugiere.

Task 4.2 - Comparación LSTM vs Transformer

Modelo	Horizonte	MAE	RMSE	Ratio vs naive
LSTM	24h	0.2026	0.2726	1.015
LSTM	48h	0.2771	0.3623	1.064
Transformer	24h	0.1910	0.2495	0.957
Transformer	48h	0.2378	0.3079	0.913
Naive	24h	0.1996	0.2672	1.000
Naive	48h	0.2604	0.3359	1.000

Pregunta a. Explicar matemáticamente por qué el error aumenta con el horizonte, tanto en el caso iterativo como en el direct multi-output implementado.

Al aumentar el horizonte de 24 a 48 horas el error de ambos modelos sube, y para el LSTM eso tiene una explicación matemática directa si pensamos en cómo se usaría de forma iterativa (recursiva): si en cada paso alimentamos la predicción anterior como si fuera la observación real, $\hat{y}_{t+k} = f(\hat{y}_{t+k-1}, h_{t+k-1})$, el error en el paso h se puede aproximar linealizando f alrededor de la trayectoria real,

$$e_h \approx \frac{\partial f}{\partial x} e_{h-1} + \varepsilon_h$$

y desenrollando la recursión queda:

$$e_H \approx \sum_{k=1}^H \left(\prod_{j=k+1}^H \left(\frac{\partial f}{\partial x} \right)_j \right) \varepsilon_k$$

el error de cada paso se arrastra y se acumula en todos los pasos siguientes, así que cuantos más pasos iteramos, más términos de ruido se suman y más grande es el error esperado en H .

Pero nuestro LSTM (y el Transformer) no funcionan así: entrenamos un modelo directo por horizonte, que ve la ventana real de 96 valores observados y predice x_{t+H} de un solo salto, sin encadenar predicciones. Ahí no hay acumulación iterativa de error y aun así el error sube con H (LSTM: MAE 0.2026 en $H=24$ vs 0.2771 en $H=48$; Transformer: 0.1910 vs 0.2378) por una razón distinta: la ventana de entrada es la misma ($L=96$) para ambos horizontes, pero la correlación lineal entre esa ventana y el valor objetivo cae con el lag, como muestra la propia ACF ($\rho(24) = 0.925$ contra $\rho(48) = 0.877$). O sea que, aunque el modelo vea exactamente la misma información de entrada, el techo estadístico de lo que esa información puede predecir es más bajo cuanto más lejos está el objetivo, así que incluso un modelo perfectamente entrenado tendría más error en $H=48$ solo porque hay menos señal explotable en la ventana para ese horizonte.

Pregunta b. Si el Transformer supera al LSTM en $H=48$ pero no en $H=24$, explicar con base en el flujo del gradiente.

En nuestros resultados el Transformer superó al LSTM en ambos horizontes (no solo en $H=48$ como plantea el enunciado como posibilidad), así que el patrón exacto no se cumple al pie de la letra, pero la tendencia sí: la ventaja del Transformer sobre el LSTM crece muchísimo con el horizonte. En $H=24$ la diferencia de ratios es $1.015 - 0.957 = 0.058$ (5.8 puntos porcentuales), y en $H=48$ es $1.064 - 0.913 = 0.151$ (15.1 puntos), o sea que la brecha casi se triplica.

Creemos que la explicación pasa por cómo fluye el gradiente en cada arquitectura: en el LSTM, el gradiente que le llega a un paso temporal lejano depende del producto de los Jacobianos de la celda a lo largo de toda la secuencia,

$$\frac{\partial c_T}{\partial c_0} = \prod_{t=1}^T f_t$$

con f_t la compuerta de olvido en cada paso, y como cada f_t es un sigmoide (estrictamente menor que 1), ese producto de 96 términos tiende a encogerse, así que la información de las posiciones más antiguas de la ventana le llega más débil a la salida. La atención, en cambio, conecta la posición 0 (CLS) con cualquier otra posición con un solo salto,

$$e_{0s} = \frac{q_0^T k_s}{\sqrt{d_k}}$$

sin productos de Jacobianos de por medio, así que el gradiente hacia cualquier posición de la ventana no se degrada con la distancia. Nos parece que esa ventaja importa poco cuando el horizonte es corto y el patrón dominante es casi puramente “el último valor” (ambos modelos lo aprenden fácil, por eso la brecha en H=24 es chica), pero se vuelve más valiosa cuando el horizonte crece y hay menos señal de corto plazo disponible, porque ahí sacarle provecho a cualquier parte de la ventana de 96 pasos (no solo a los últimos) importa más, y el Transformer está en mejores condiciones estructurales para hacerlo que el LSTM.